

Deploying ML Models on AWS Lambda with ECR

A Step-by-Step Serverless Deployment Guide

Prabal

April 20, 2026

Contents

1	Introduction	2
2	Architecture Overview	2
3	Lambda Constraints (Read First)	2
4	Prerequisites	3
5	Step-by-Step Deployment	3
5.1	Step 1 — Project Layout	3
5.2	Step 2 — Write the Lambda Handler	3
5.3	Step 3 — Write the Dockerfile	4
5.4	Step 4 — Create an ECR Repository	4
5.5	Step 5 — Build, Tag, Push the Image	5
5.6	Step 6 — Create the Lambda Execution Role	5
5.7	Step 7 — Create the Lambda Function	5
5.8	Step 8 — Expose via HTTP	6
5.9	Step 9 — Invoke	6
5.10	Step 10 — Update and Roll Back	6
6	Dealing with Cold Starts	7
7	Externalizing Model Weights	7
8	Framework-Specific Notes	8
8.1	PyTorch	8
8.2	HuggingFace Transformers	8
8.3	ONNX Runtime	8
8.4	scikit-learn / XGBoost / LightGBM	8
9	Observability	8
10	Cost Model	9
11	Production Best Practices	9
12	Troubleshooting Checklist	9
13	Summary	10

1 Introduction

AWS Lambda is a serverless compute service that runs code on demand without provisioning or managing servers. When combined with Amazon Elastic Container Registry (ECR), Lambda can execute **container images up to 10 GB**, which is large enough to host many small-to-medium ML models along with their inference runtime (PyTorch, ONNX, scikit-learn, HuggingFace Transformers).

This deployment pattern — often called “*Lambda + ECR*” — is the most cost-effective way to serve ML models that receive **sporadic or unpredictable traffic**. You pay only for the milliseconds during which your function is actually executing; idle endpoints cost nothing.

Note

When to prefer Lambda + ECR over SageMaker:

- Model fits in ≤ 10 GB image and ≤ 10 GB memory.
- Traffic is spiky, infrequent, or unpredictable (hours of idle per day).
- Inference latency tolerance is moderate (cold starts can be seconds).
- No GPU required (Lambda is CPU-only).
- You want tight integration with API Gateway, S3 events, EventBridge, SQS, Step Functions, etc.

2 Architecture Overview

The typical topology:

Client → API Gateway → Lambda (container) → Response

with the container image pulled from ECR at cold start, and model weights optionally cached from S3 or baked directly into the image.

Key components:

- **ECR** — private Docker registry that stores the inference image.
- **Lambda** — executes the container on demand; reuses warm containers across invocations.
- **API Gateway / Function URL** — HTTPS entry point.
- **IAM role** — Lambda execution role granting permission to pull from ECR and read model artifacts from S3 (if externalized).
- **CloudWatch Logs & Metrics** — observability.

3 Lambda Constraints (Read First)

Before committing to this path, confirm your model fits within Lambda’s limits:

Resource	Hard limit
Container image size	10 GB (uncompressed, as stored in ECR)
Memory	128 MB – 10 240 MB (10 GB)
vCPU	Scales linearly with memory (6 vCPUs at 10 GB)
Execution timeout	15 minutes (900 s)
/tmp scratch space	512 MB – 10 GB (configurable)
Request / response payload	6 MB sync, 256 KB async (API Gateway: 10 MB)

Concurrent executions	1 000 per region by default (soft limit)
GPU	Not supported

Important

If your model weights + runtime exceed **10 GB**, or if you need GPU inference, Lambda is the wrong tool — use SageMaker, ECS/Fargate, or EC2 instead.

4 Prerequisites

1. AWS account, AWS CLI configured (`aws configure`).
2. Docker installed and running locally.
3. Python 3.9+.
4. A trained model artifact (e.g., `model.pkl`, `model.onnx`, `model.pt`, or a HuggingFace checkpoint folder).
5. IAM permissions to create ECR repos, Lambda functions, and IAM roles.

5 Step-by-Step Deployment

5.1 Step 1 — Project Layout

A minimal layout:

```
lambda-ml/
  app.py           # Lambda handler
  requirements.txt  # Python dependencies
  model/           # model weights (baked into image)
    model.pkl
  Dockerfile
```

5.2 Step 2 — Write the Lambda Handler

The handler loads the model *once at module scope* so it survives across warm invocations. Loading inside `lambda_handler` would pay the cost on every request.

```
1 import json
2 import pickle
3 import os
4 import numpy as np
5
6 # ---- Cold-start initialization (runs once per container) ----
7 MODEL_PATH = os.environ.get("MODEL_PATH", "/var/task/model/model.pkl")
8 with open(MODEL_PATH, "rb") as f:
9     MODEL = pickle.load(f)
10
11 def lambda_handler(event, context):
12     try:
13         # API Gateway proxy integration wraps the payload in "body"
14         body = event.get("body", event)
15         if isinstance(body, str):
16             body = json.loads(body)
17
18         features = np.array(body["inputs"], dtype=np.float32)
```

```

19     preds = MODEL.predict(features).tolist()
20
21     return {
22         "statusCode": 200,
23         "headers": {"Content-Type": "application/json"},
24         "body": json.dumps({"predictions": preds}),
25     }
26 except Exception as e:
27     return {
28         "statusCode": 500,
29         "body": json.dumps({"error": str(e)}),
30     }

```

Tip

The model is loaded at import time so it is cached across invocations on the same warm container. AWS keeps warm containers alive for ~5–15 minutes of inactivity, and longer under sustained traffic.

5.3 Step 3 — Write the Dockerfile

Use the official AWS Lambda Python base image — it ships the Lambda Runtime Interface Client (RIC) pre-installed, so your container already speaks the Lambda invocation protocol.

```

# Base image maintained by AWS (Python 3.11)
FROM public.ecr.aws/lambda/python:3.11

# Install Python dependencies into the Lambda task root
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt \
    --target "${LAMBDA_TASK_ROOT}"

# Copy code and model weights
COPY app.py          ${LAMBDA_TASK_ROOT}/
COPY model/          ${LAMBDA_TASK_ROOT}/model/

# Tell Lambda which function to invoke: file.function
CMD ["app.lambda_handler"]

```

Example requirements.txt:

```

numpy==1.26.4
scikit-learn==1.4.2

```

Important

The base image is linux/amd64. If you are on an Apple Silicon Mac, build with `-platform linux/amd64` or the image will fail to start in Lambda with cryptic `exec format error`.

5.4 Step 4 — Create an ECR Repository

```

export AWS_REGION=us-east-1
export ACCOUNT_ID=$(aws sts get-caller-identity --query Account --output text)
export REPO_NAME=lambda-ml-inference

aws ecr create-repository \
    --repository-name $REPO_NAME \
    --region $AWS_REGION \
    --image-scanning-configuration scanOnPush=true

```

5.5 Step 5 — Build, Tag, Push the Image

```
# 1. Authenticate Docker to ECR
aws ecr get-login-password --region $AWS_REGION | \
    docker login --username AWS \
        --password-stdin $ACCOUNT_ID.dkr.ecr.$AWS_REGION.amazonaws.com

# 2. Build (force linux/amd64 for Lambda)
docker build --platform linux/amd64 -t $REPO_NAME:v1 .

# 3. Tag for ECR
docker tag $REPO_NAME:v1 \
    $ACCOUNT_ID.dkr.ecr.$AWS_REGION.amazonaws.com/$REPO_NAME:v1

# 4. Push
docker push $ACCOUNT_ID.dkr.ecr.$AWS_REGION.amazonaws.com/$REPO_NAME:v1
```

Tip

Use immutable, version-tagged images (v1, v2, git SHA) rather than `latest`. Lambda pins the image digest at function creation, so overwriting `latest` does not actually roll out a new version — and worse, makes rollbacks ambiguous.

5.6 Step 6 — Create the Lambda Execution Role

Trust policy (trust-policy.json):

```
{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Allow",
    "Principal": {"Service": "lambda.amazonaws.com"},
    "Action": "sts:AssumeRole"
  }]
}
```

Create the role and attach the basic execution policy:

```
aws iam create-role \
    --role-name lambda-ml-role \
    --assume-role-policy-document file://trust-policy.json

aws iam attach-role-policy \
    --role-name lambda-ml-role \
    --policy-arn arn:aws:iam::aws:policy/service-role/
    AWSLambdaBasicExecutionRole
```

Attach extra permissions if the function needs to read weights from S3:

```
aws iam attach-role-policy \
    --role-name lambda-ml-role \
    --policy-arn arn:aws:iam::aws:policy/AmazonS3ReadOnlyAccess
```

5.7 Step 7 — Create the Lambda Function

```
aws lambda create-function \
    --function-name ml-inference \
    --package-type Image \
    --code ImageUri=$ACCOUNT_ID.dkr.ecr.$AWS_REGION.amazonaws.com/$REPO_NAME:v1
    \
```

```
--role arn:aws:iam::$ACCOUNT_ID:role/lambda-ml-role \
--timeout 60 \
--memory-size 3008 \
--ephemeral-storage Size=2048 \
--architectures x86_64 \
--region $AWS_REGION
```

Note

Memory sizing is critical. Lambda scales vCPU linearly with memory — more memory often makes the function *both faster and cheaper* per invocation because billed duration drops. For ML inference, start at **3 008 MB (2 vCPU)** and benchmark up/down from there.

5.8 Step 8 — Expose via HTTP

Two simple options:

Option A — Function URL (quickest).

```
aws lambda create-function-url-config \
  --function-name ml-inference \
  --auth-type NONE      # or AWS_IAM for signed requests

aws lambda add-permission \
  --function-name ml-inference \
  --statement-id FunctionURLAllowPublicAccess \
  --action lambda:InvokeFunctionUrl \
  --principal "*" \
  --function-url-auth-type NONE
```

Returns a URL like: `https://<random>.lambda-url.us-east-1.on.aws/`

Option B — API Gateway (production-grade). Provides throttling, custom domains, WAF, usage plans, and request/response transformation. Attach an HTTP API with a Lambda proxy integration pointing to the function ARN.

5.9 Step 9 — Invoke

```
curl -X POST https://<function-url>/ \
  -H "Content-Type: application/json" \
  -d '{"inputs": [[0.1, 0.4, 0.9]]}'
```

Or from Python:

```
1 import boto3, json
2 lam = boto3.client("lambda", region_name="us-east-1")
3 resp = lam.invoke(
4     FunctionName="ml-inference",
5     Payload=json.dumps({"inputs": [[0.1, 0.4, 0.9]]}),
6 )
7 print(json.loads(resp["Payload"]).read())
```

5.10 Step 10 — Update and Roll Back

Push a new tag, then point Lambda at the new digest:

```

docker build --platform linux/amd64 -t $REPO_NAME:v2 .
docker tag $REPO_NAME:v2 \
    $ACCOUNT_ID.dkr.ecr.$AWS_REGION.amazonaws.com/$REPO_NAME:v2
docker push $ACCOUNT_ID.dkr.ecr.$AWS_REGION.amazonaws.com/$REPO_NAME:v2

aws lambda update-function-code \
    --function-name ml-inference \
    --image-uri $ACCOUNT_ID.dkr.ecr.$AWS_REGION.amazonaws.com/$REPO_NAME:v2

```

Use **versions and aliases** plus **CodeDeploy** for canary / linear rollouts with automatic rollback on CloudWatch alarm.

6 Dealing with Cold Starts

Cold start = the time Lambda spends pulling the image, initializing the container, and running module-level code (including model load). For ML images this is often **2–15 seconds**, which can dominate tail latency.

Mitigations, in order of impact:

1. **Provisioned Concurrency** — reserves a pool of always-warm containers; eliminates cold starts entirely within that pool. Increases cost (billed hourly for reserved capacity).
2. **SnapStart for Python / containers** — caches initialized state snapshots; dramatically reduces cold-start for supported runtimes.
3. **Smaller images** — prune dev dependencies; use slim base images; quantize or compress model weights.
4. **Lazy imports** — only `import torch` etc. inside the functions that need them if the handler has multiple code paths.
5. **Right-size memory upward** — more memory = more CPU = faster init. Counter-intuitively often *cheaper* because duration drops.
6. **Pre-warming pings** — EventBridge rule hitting the function every few minutes. Cheap and cheerful but not guaranteed.

Tip

For customer-facing synchronous APIs on spiky traffic, **Provisioned Concurrency** is usually the right answer. Reserve 1–2 instances; it is inexpensive compared to a SageMaker endpoint running 24/7.

7 Externalizing Model Weights

Baking weights into the image is simplest but rebuilds on every model update. For faster iteration or for models that approach the 10 GB image limit, store weights in S3 and fetch them at cold start into `/tmp`:

```

1 import os, boto3, pickle
2
3 BUCKET = os.environ["MODEL_BUCKET"]
4 KEY     = os.environ["MODEL_KEY"]
5 LOCAL  = "/tmp/model.pkl"
6
7 if not os.path.exists(LOCAL):

```

```

8 boto3.client("s3").download_file(BUCKET, KEY, LOCAL)
9
10 with open(LOCAL, "rb") as f:
11     MODEL = pickle.load(f)

```

Note

`/tmp` persists across warm invocations but *not* across cold starts. Each new container will re-download. For multi-GB weights, this can add seconds to cold start — weigh the tradeoff against image-based loading. For shared model storage across many Lambda containers, consider mounting Amazon EFS.

8 Framework-Specific Notes

8.1 PyTorch

- Install the *CPU-only* wheel: `torch --index-url https://download.pytorch.org/whl/cpu` — avoids CUDA bloat and keeps the image under the 10 GB limit.
- Call `model.eval()` and wrap inference in `torch.inference_mode()`.
- Consider exporting to TorchScript or ONNX for faster load and inference.

8.2 HuggingFace Transformers

- Set `TRANSFORMERS_CACHE=/tmp` to avoid writes into the read-only image layer.
- Pre-download weights at build time with a `RUN python -c "..."` step rather than fetching from the Hub at cold start.
- For large models (7B+) prefer quantized GGUF / int4 variants or skip Lambda entirely.

8.3 ONNX Runtime

- Excellent fit: small runtime, fast CPU inference, low cold start.
- Use `onnxruntime` (not `onnxruntime-gpu`) for Lambda.

8.4 scikit-learn / XGBoost / LightGBM

- Near-ideal fit — small dependencies, fast cold start, cheap.
- Pickle version pinning matters: load with the same library version that pickled the model.

9 Observability

- **CloudWatch Logs** — all `stdout/stderr` lands in `/aws/lambda/<function-name>`. Use structured JSON logs for easy querying with CloudWatch Logs Insights.
- **CloudWatch Metrics** — `Invocations`, `Errors`, `Throttles`, `Duration`, `ConcurrentExecutions`, `IteratorAge` (streams).
- **AWS X-Ray** — enable active tracing for per-segment latency breakdown (init vs. invoke vs. downstream calls).
- **Lambda Insights** — enhanced metrics (CPU, memory, network, disk) via a bundled extension layer.

10 Cost Model

Lambda pricing has two components per invocation:

1. **Requests** — \$0.20 per 1 M requests.
2. **Duration** — billed per ms, proportional to memory allocated (GB-seconds).

Back-of-envelope example. A function at 3 008 MB averaging 500 ms per invocation, 1 M invocations per month:

$$\text{Durationcost} \approx 1,000,000 \times 0.5 \text{ s} \times \frac{3008}{1024} \text{ GB} \times \$0.0000166667/\text{GB} \cdot \text{s} \approx \$24.50$$

plus \$0.20 requests = ~\$24.70/month. Compared to a `ml.m5.xlarge` SageMaker endpoint at ~\$140/month running 24/7, this is ~6× cheaper for the same invocation count — but SageMaker wins once traffic is high enough to keep the endpoint continuously busy.

Tip

The crossover point where SageMaker becomes cheaper is typically around **5–10 million invocations per month** for CPU workloads. Below that, Lambda wins; above that, a right-sized persistent endpoint wins.

11 Production Best Practices

1. **Infrastructure as Code** — Define everything (ECR repo, Lambda, IAM role, API Gateway) in AWS SAM, CDK, or Terraform.
2. **CI/CD** — GitHub Actions / CodePipeline: build image → push to ECR → update function → shift alias with CodeDeploy canary.
3. **Versions & aliases** — Publish immutable versions; point `prod` and `staging` aliases at specific versions for safe rollouts.
4. **Idempotency** — For async / SQS-triggered inference, de-duplicate on a request ID; Lambda may retry on failure.
5. **Reserved / Provisioned Concurrency** — Protect critical functions from noisy-neighbor throttling; eliminate cold starts.
6. **VPC only when needed** — VPC-attached Lambdas add ENI cold-start overhead. Attach only if you must reach RDS / ElastiCache / private subnets. Use VPC endpoints to avoid NAT Gateway charges.
7. **Secrets** — Use Secrets Manager or SSM Parameter Store, not environment variables, for sensitive values.
8. **Image hygiene** — Multi-stage builds; strip tests and docs; `pip install --no-cache-dir`; remove `__pycache__`. Target < 2 GB images when possible.

12 Troubleshooting Checklist

Symptom	Likely Cause / Fix
---------	--------------------

<code>exec format error</code> at cold start	Image built for wrong architecture. Rebuild with <code>-platform linux/amd64</code> .
<code>Runtime.InvalidEntrypoint</code>	CMD in Dockerfile does not match handler. Must be <code>["file.function"]</code> (no <code>.py</code>).
Cold start > 10 s	Image too large / model too large. Increase memory, use Provisioned Concurrency, externalize weights to EFS, or switch to SageMaker.
Task timed out after 3.00 seconds	Default timeout is 3 s. Raise with <code>-timeout</code> (max 900).
<code>[Errno 28] No space left on device</code>	Writing to <code>/tmp</code> which defaults to 512 MB. Increase ephemeral storage or write elsewhere.
<code>AccessDeniedException</code> pulling image	Lambda execution role needs ECR permissions, <i>or</i> the ECR repo policy must allow the Lambda service principal.
Memory climbs invocation-over-invocation	Module-level state accumulating. Clear caches in the handler or recycle containers by bumping a no-op env var.

13 Summary

The canonical Lambda + ECR deployment flow is:

1. Write handler with module-level model load.
2. Write Dockerfile using `public.ecr.aws/lambda/python`.
3. Create ECR repository.
4. Build (`-platform linux/amd64`), tag, push.
5. Create IAM execution role.
6. Create Lambda function with `-package-type Image`.
7. Expose via Function URL or API Gateway.
8. Tune memory, timeout, ephemeral storage.
9. Attach Provisioned Concurrency for latency-sensitive paths.
10. Monitor with CloudWatch, X-Ray, Lambda Insights.

Lambda + ECR is the right answer when your traffic is spiky, your model is small-to-medium, and you want to pay nothing when idle — which describes a surprising fraction of real-world ML workloads.